

SMART API

Backend para flujo digital de restaurantes

Descripción

Proyecto Java 25 Backend para la digitalización de restaurantes.

API REST para menú, pedidos, pagos, cocina y administración en un restaurante.

Enfoque del proyecto

Desarrollado como estudiante de backend,

centrado en seguridad, flujo y práctica real, mediante requests.http para la prueba en distintos roles.

Java

Spring Boot

Maven

JPA

Spring Security

JWT

MySQL

Spring AI

OpenAI

Ollama

MCP

JFreeChart

ZXing

Francisco Manuel Candela Manchón

Ironhack Java Backend Final Project



Enlace al
repositorio

Sobre mí

Un poco sobre mí

Soy de Crevillente una localidad de la provincia de Alicante, y actualmente vivo en Toledo capital.

Me interesa mucho la tecnología y, fuera del ordenador, disfruto del Senderismo con la compañía de mis perritas, y de vez en cuando hago alguna salida de MTB.

Estudiante de Java Back-end, todavía con muchas cosas por aprender .
Y con ganas de seguir formándome en el sector, con la futura trayectoria de aprende Front-end.

Crevillente



Mi punto de partida.

Cerca de la sierra y la naturaleza.

Toledo



Donde sigo formándome

y preparando mi camino.

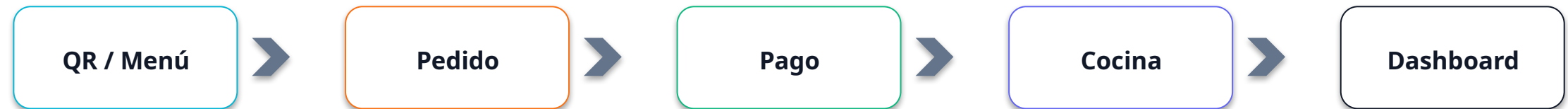
De una carta QR a un flujo completo

El problema

En muchos restaurantes, el QR solo abre una carta en PDF.
El pedido y la gestión siguen siendo manuales.

La solución

SmartQR lleva esa idea un paso más allá.
Conecta menú, pedido, pago, cocina y administración.



Tres perfiles, tres responsabilidades

CUSTOMER

Carta, pedidos y pagos.
Estado de pedido, ticket,
feedback y combos
generados por IA a petición
del cliente.

EMPLOYEE

Crear un QR de mesa.
Ver la cola de cocina, estados
de pedido y validar los
pagos en efectivo.

ADMIN

Gestión de productos.
Métricas, gráficos,
Ollama/MCP y chat con
OpenIA.

La demo recorre el flujo por perfiles: primero cliente, después empleado y por último administrador.

Arquitectura y seguridad

Capas del proyecto

Controller

DTO

Service

Repository

Entity

MySQL

Seguridad

- Spring Security con JWT
- Roles: CUSTOMER, EMPLOYEE y ADMIN
- Control de pedidos propios del cliente

No basta con estar autenticado: también hay que comprobar a qué datos puede acceder cada usuario.

Retos técnicos más importantes

Reto 1

Separar bien los flujos
por perfil y ordenar las
rutas para que todo tuviera
sentido.

JWT

Roles

Reto 2

Combinar login con JWT,
permisos por rol y control
de pedidos propios.

Ownership

Cómo lo resolví

Rutas protegidas,
por rol y servicios validando
reglas
de negocio antes de tocar
datos.

Servicios

Validaciones

Errores y aprendizaje

Errores

- Confiar demasiado en tener login con JWT y no revisar bien el ownership al principio.
- Dejar algunas validaciones del flujo para demasiado tarde, ya en fase de pruebas.
- Intentar meter varias cosas a la vez en lugar de cerrar antes el flujo base.

Qué aprendí

- La seguridad no es solo autenticación: también es control de acceso a datos reales.
- El flujo principal hay que probarlo pronto, aunque todavía no esté todo perfecto.
- Como estudiante de backend, me compensa más una base sólida que añadir extras demasiado rápido.

DEMO

Flujo principal de la API

Entorno de ejecución en IntelliJ IDEA en archivos .http separados por ROL.

CUSTOMER

- Obtener el menú
- Crear un pedido
- Simular un pago con tarjeta
- Obtener el ticket del pedido después de procesar el pago
- Enviar un feedback
- EXTRA: Obtener una recomendación de combo para "X" comensales y precio limite, gracias al API de OpenAI

EMPLOYEE

- Gestionar los pedidos en cocina
- Cambiar el estado de un pedido
- Validar los pagos en efectivo

ADMIN

- Ver el dashboard
- Obtener los gráficos de ventas
- Obtener una respuesta de la IA referente a los feedbacks por su estado, POSITIVE, NEGATIVE o NEUTRAL
- LLM local para consultas, READ ONLY
- Probar chat OpenIA con memoria

CUSTOMER_REQUESTS.http

EMPLOYEE_REQUESTS.http

ADMIN_REQUESTS.http

DEMO_REQUESTS.http

Mejoras futuras

- Añadir un frontend para que el flujo se vea completo de cara al usuario.
- Mejorar la estimación visual del tiempo de espera del pedido propio.
- Ampliar el dashboard con más filtros e histórico de métricas.
- Reforzar tests y validaciones para cubrir mejor el flujo completo.

REST API

Gestión de restaurante por QR

< > REST API

¡Gracias por su atención!



SmartQr

```
<?xml version="1.0"?>
<Rest>
  <!DOCTYPE Person [
    <!-- HERE I GO -->
  ] -->
</Rest>
```

```
<?xml version="1.0"?>
<?xml encoding="UTF-8"?>
<?xml namespace="http://www.w3.org/2001/XMLSchema-instance"?>
<?xml schemaLocation="http://www.w3.org/2001/XMLSchema-instance#xsi" type="Person"?>
<Person>
  <Name>Sakur</Name>
  <Address>Sakur</Address>
  <Phone>Sakur</Phone>
</Person>
```



fco.manuel.candela90@gmail.com



github.com/fcomanuelcandela90-dev/SmartQR_API

Escanear
para
contactar



Transformando la experiencia del restaurante.